

Codefest 7

Hardware for Artificial Intelligence and Machine Learning (HW4AI)

ECE 410/510

Spring 2026

CMAN — Sparsity breakeven analysis

A dense *Matrix-Vector Multiply* (MVM) on an $N \times N$ weight matrix W performs N^2 MACs and loads N^2 FP32 weights from memory. Sparse computation using *Compressed Sparse Row* (CSR) format stores only non-zero weights but adds index overhead. CSR storage uses three arrays: a values array (one FP32 weight per non-zero, 4 bytes each), a column index array (one INT32 per non-zero, 4 bytes each), and a row pointer array of length $N+1$ (one INT32 per row pointer, 4 bytes each). Dense storage: 4 bytes per element.

Tasks

- Let $N=512$ and sparsity s (fraction of zeros). Write expressions for:
 - dense MVM compute (FLOPs);
 - dense memory bytes;
 - sparse compute (FLOPs, as a function of s);
 - sparse memory bytes (as a function of s).
- Compute the theoretical speedup of sparse vs. dense MVM (FLOPs ratio) as a function of s . At what sparsity s is the FLOPs speedup equal to $2\times$?
- Compute the sparsity level s at which sparse memory bytes = dense memory bytes (the memory breakeven). Show the derivation. Above this sparsity, sparse format uses less memory.
- If hardware perfectly exploits sparsity (skips zero MACs and their memory loads), compute the end-to-end execution time speedup at $s=0.9$ for a memory-bandwidth-limited system with 320 GB/s bandwidth. Use $N=512$.

CMAN NO AI

Deliverable:

codefest/cf07/cman_sparsity_analysis.md (or .pdf/.png) containing

- four expressions for dense and sparse compute/memory;
- FLOPs speedup expression and s for $2\times$ speedup;
- memory breakeven sparsity with derivation;
- end-to-end speedup at $s=0.9$ for memory-bound case.

CLLM — OpenLane synthesis interpretation

Run an OpenLane 2 synthesis on your SystemVerilog compute core and use an LLM to help interpret the results. This codefest is direct preparation for M3 (due May 24), which requires a synthesis attempt. The goal is to confront the actual numbers. i.e., critical path, slack, area, warnings, before M3 rather than during the final week.

Tasks

- Choose a synthesis target. **Option A** (preferred if your project core is ready): the compute core for your project. **Option B** (fallback): the corrected 4×4 crossbar MAC from CF06 (`crossbar_mac.sv`). Copy your chosen file to `codefest/cf07/hdl/synth_top.sv`. Run OpenLane 2 on it. Commit the key reports (`final/reports/metrics.csv` and the synthesis and STA reports) to `codefest/cf07/synth/`.
- Use an LLM to help interpret the synthesis reports. Write `codefest/cf07/synth/synth_interpretation.md` (200–300 words) covering:
 - the clock period you ran at and the worst-case slack;
 - the critical path: source register, sink register, dominant cell types along the path;
 - total cell area and the top three contributors by area or instance count;

- (d) any failed constraints, hold violations, or warnings worth investigating. Be specific to the numbers your synthesis produced; do not paste generic LLM commentary.
3. Write `codefest/cf07/synth/m3_plan.md` (100–150 words).
 - a. If you synthesized your own project core (**Option A**): state what you will change for M3, e.g., pipeline the critical path, drop precision, reduce unrolling, change clock target, or keep as-is, with each decision grounded in a specific number from your synthesis report.
 - b. If you synthesized the CF06 fallback (**Option B**): state when you will attempt synthesis on your actual project core (target date before M3), what you expect to be different (size, critical path location, precision), and what you learned from the fallback exercise that you will apply.
 4. Project scope assessment: update `project/scope_assessment.md` confirming or adjusting your project scope, with a brief rationale grounded in your synthesis result and your M3 plan.

CLLM OK	AI Deliverable: GitHub commit containing • <code>codefest/cf07/hdl/synth_top.sv</code>; • <code>codefest/cf07/synth/</code> folder with OpenLane 2 reports (<code>metrics.csv</code>, synthesis and STA reports); • <code>codefest/cf07/synth/synth_interpretation.md</code> (200–300 words covering clock/slack, critical path, area, warnings); • <code>codefest/cf07/synth/m3_plan.md</code> (100–150 words); • <code>project/scope_assessment.md</code> updated.
------------	---